Note 3: Maven Build Lifecycle Phases

What is a Build Lifecycle?

A Maven build lifecycle is a well-defined sequence of phases that describes the process of building and distributing an artifact. When you tell Maven to execute a particular phase, it executes every phase that comes before it in the sequence as well.

Maven has three built-in lifecycles:

- default — handles the main build and deployment process
- clean — handles project cleaning (deleting the target/ folder)
- site — handles the creation of project documentation

The most important is the default lifecycle.

_____

Default Lifecycle Phases (in order)

1. validate Validates that the project is correct and all necessary information is available. Checks that the pom.xml is well-formed and that all required fields are present.

mvn validate

2. compile Compiles the project's source code. Maven reads all .java files from src/main/java/, compiles them using the Java compiler, and places the resulting .class files in target/classes/.

mvn compile

3. test Runs the unit tests using a testing framework (JUnit or TestNG). Test classes are compiled first and placed in target/test-classes/. Maven uses the Surefire plugin to run the tests and generate reports in target/surefire-reports/. If any test fails, the build fails at this phase by default.

mvn test

4. package Takes the compiled code and packages it into a distributable format — a .jar or .war file — and places it in the target/ directory. The output file is named using the artifactId and version from the POM: my-app-1.0.0.jar.

mvn package

5. verify Runs any checks to verify the package is valid and meets quality criteria. This is where integration tests are typically run. Integration tests test the interaction between components, unlike unit tests which test components in isolation.

mvn verify

6. install Installs the packaged artifact (the .jar or .war) into the local Maven repository, located at ~/.m2/repository/ on your machine. Once installed, other Maven projects on the same machine can use this artifact as a dependency by referencing its groupId, artifactId, and version.

mvn install

7. deploy Copies the final artifact to a remote repository (such as Nexus, Artifactory, or Maven Central) so that other developers and build servers can access it. This is typically only run in a CI/CD pipeline.

mvn deploy

How Phase Sequencing Works

This is one of Maven's most important behaviours. When you run a phase, Maven runs all preceding phases first.

Examples:

- mvn package → runs validate → compile → test → package
- mvn install → runs validate → compile → test → package → verify → install
- mvn test → runs validate → compile → test

This means you never need to manually chain commands. Running mvn install is enough to compile, test, package, and install.

The Clean Lifecycle

The clean lifecycle has three phases:

- pre-clean — executes processes needed before the actual cleaning
- clean — removes the target/ directory and all generated files
- post-clean — executes processes needed after the cleaning

mvn clean

It is very common to combine clean with a default lifecycle phase to ensure a fresh build:

mvn clean install

mvn clean package

mvn clean test

Running mvn clean install deletes all previous build output and then runs the full build from scratch. This prevents stale compiled files from causing inconsistent builds.

Skipping Tests

Sometimes during development you want to package quickly without running tests:

mvn package -DskipTests

Or to skip both compiling and running tests:

mvn package -Dmaven.test.skip=true

This should never be done in production CI/CD pipelines — it is only acceptable for local development speed.

Summary of Lifecycle Phases

Phase What it Does Output Location

validate Checks POM validity —

compile Compiles source code target/classes/

test Runs unit teststarget/surefire-reports/

package Creates JAR/WAR target/my-app-1.0.0.jar

verify Runs integration tests —

install Copies to local repo ~/.m2/repository/

deploy Copies to remote repo Remote registry

clean Deletes build output Removes target/